

From Probabilistic Verification to Executable Contracts and Machine- Checked Quantum Circuits

Introduction to Quantum Computing
Leonardo Chiaretti

Motivation and Scope

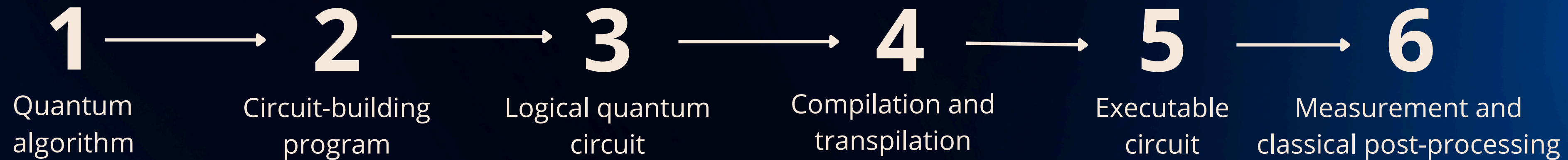
Why study formal validation for quantum software?

- Quantum programs are difficult to inspect and debug.
- Measurement reveals only partial information about a quantum state and changes its post-measurement state.
- Simulation checks finite instances, not general correctness.
- Compilation may significantly transform the original circuit.
- Different techniques provide different levels of assurance.

We focus primarily on semantic correctness of quantum software.

Syntax, type systems, and static analysis are considered as preliminary layers enabling formal verification.

From Algorithm to Physical Execution



Why Testing Is Not Enough

“Program testing can be used to show the presence of bugs, but never to show their absence!”
- Dijkstra

Testing provides evidence that a property holds for selected executions.: $P(x_1), P(x_2), \dots, P(x_k)$

Universal correctness property: $\forall x \in D, P(x)$

Meaning: for every input x belonging to the domain D , the property P holds.

What Does “Validation” Mean?

Technique	Main question
Type checking and static analysis	Is the program well formed and statically safe?
Testing	Does the program work on these inputs?
Simulation	What state or distribution does the model produce?
Statistical validation	Are observed frequencies compatible with the specification?
Model checking	Does a finite-state model satisfy a temporal or quantitative property?
Equivalence checking	Do two circuits implement the same transformation?
Theorem proving	Can correctness be derived from formal definitions and rules?

Classical Foundations

Hoare-style correctness

$$\{P\} C \{Q\}$$

Where:

- (P): precondition
- (C): program;
- (Q): postcondition.

Related ideas:

- loop invariants;
- weakest preconditions;
- proof obligations;
- Design by Contract;
- specification languages such as JML.

Starting from P, every terminating execution of C must establish Q.

From Boolean to Quantitative Properties

Probabilistic programs require quantitative specifications

Bounded probability of failure:

$$\text{Pr}[\text{failure}] \leq \varepsilon$$

Almost-sure termination:

$$\text{Pr}[\text{termination}] = 1$$

Bound on expected execution time: $E[T] \leq k$

Where $E[T]$ denotes the expected value of the random variable T .

Properties may concern:

- success probabilities;
- expected running time;
- expected resource consumption;
- probabilistic termination;
- reachability probabilities.

Verification of Probabilistic Programs

Probabilistic model checking

- finite-state probabilistic models;
- reachability and temporal properties;
- tools such as PRISM and Storm.

Deductive quantitative verification

- program-oriented reasoning;
- expectation transformers;
- verification conditions;
- tools such as Caesar.

Representative Tools for Probabilistic Verification

PRISM

Approach: probabilistic model checking

Models: DTMCs, CTMCs, MDPs

Properties: reachability probabilities, temporal properties, costs and rewards

Typical question:

What is the maximum probability of eventually reaching failure?

Storm

Approach: high-performance probabilistic model checking

Inputs: PRISM and JANI models

Models: DTMCs, CTMCs, MDPs, Markov automata and POMDPs

Main feature: scalable analysis and Python integration

Caesar

Approach: deductive verification of probabilistic programs

Input: program plus quantitative specification

Method: expectation-based reasoning and verification conditions

Backend: proof obligations discharged with Z3

Example: Repeated Coin Toss

count := 0

repeat:

count := count + 1

coin := toss()

until coin = heads



$$\Pr[\text{count} = 1] = 1/2$$

$$\Pr[\text{count} = 2] = 1/4$$

$$\Pr[\text{count} = 3] = 1/8$$

...

$$\Pr[\text{count} = n] = 1/2^n$$



Verified Properties:

Almost-sure termination: $\Pr[\text{termination}] = 1$

Expected number of tosses: $E[\text{count}] = 2$

No finite worst-case bound

But, almost-sure termination does not mean termination within a fixed number of steps!

Quantum is not (only) Probabilistic

Classical probabilistic state:

$$p = (p_0, \dots, p_{n-1})$$

with:

$$p_i \geq 0 \text{ and } \sum_i p_i = 1$$

Quantum pure state:

$$|\psi\rangle = \sum_i \alpha_i |i\rangle$$

with:

$$\alpha_i \in \mathbb{C} \text{ and } \sum_i |\alpha_i|^2 = 1$$

Where:

α_i is a complex amplitude;

$|\alpha_i|^2$ is the probability of observing the basis state $|i\rangle$.

The difference lies in the amplitudes

- complex phase;
- interference;
- superposition;
- entanglement.

What Can Be Reused?

Ideas inherited from probabilistic verification:

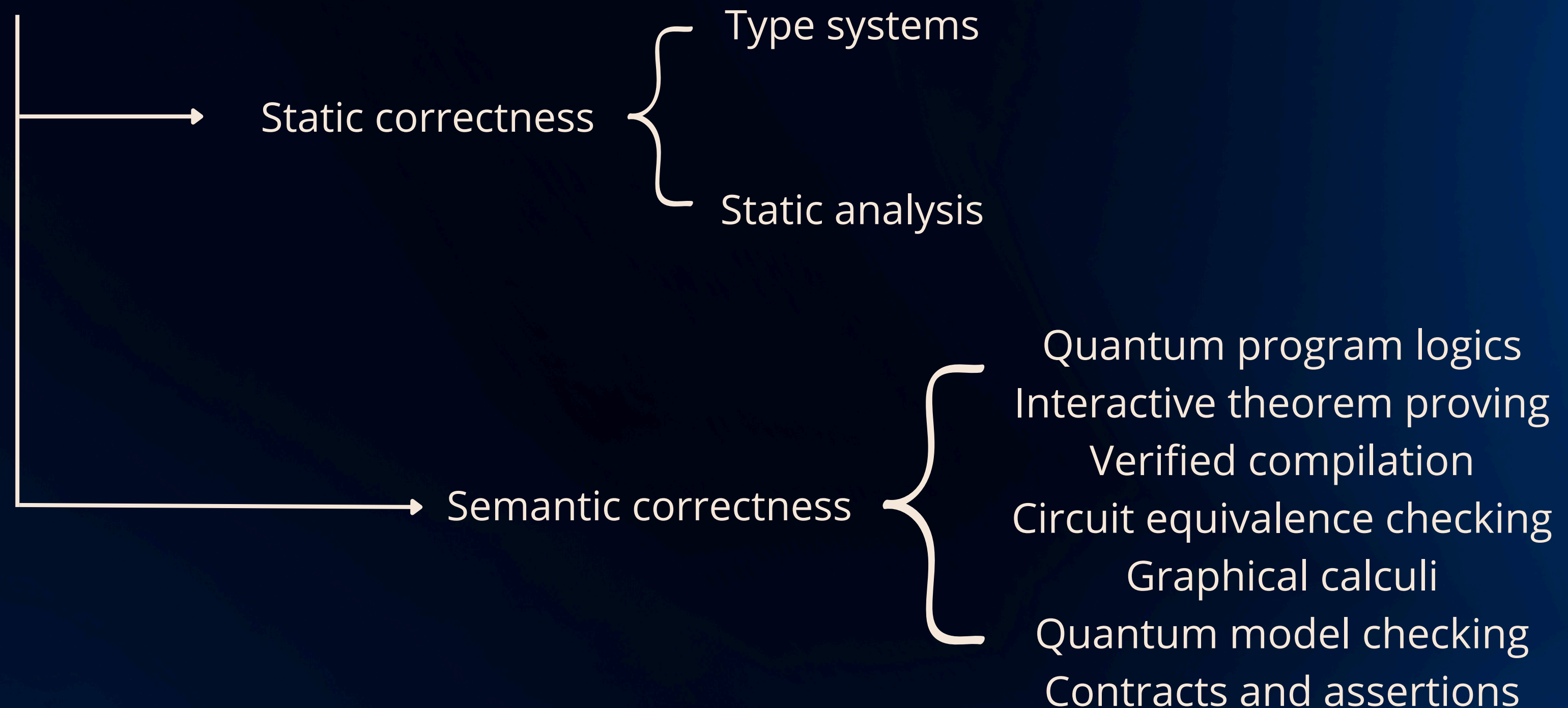
- quantitative postconditions
- success-probability bounds
- expected values
- almost-sure termination
- classical probabilistic control
- compositional reasoning

New requirements:

- state transformations by unitary operators
- density operators and quantum channels
- measurement-induced state changes
- non-commutative operations
- entangled subsystems

A Taxonomy of Quantum Software Validation

Quantum software validation



Quantum Hoare Logic

General form

$\{P\} C \{Q\}$

Same as the classical one, but, depending on the formalism, quantum assertions P and Q may be represented as:

- subspaces;
- projection operators;
- Hermitian observables;
- quantum effects;
- density operators;
- classical-quantum states.

Main objectives

- partial correctness;
- total correctness;
- reasoning about measurement and loops.

Syntax Before Verification

an illustrative core syntax:

$C ::= \text{skip}$		skip: no operation
$\mid U(q)$		$U(q)$: application of a unitary operation
$\mid C_1 ; C_2$	$\longrightarrow$	$C_1 ; C_2$: sequential composition
$\mid C_1 \otimes C_2$		$C_1 \otimes C_2$: parallel composition on disjoint registers
$\mid \text{measure}(q)$		measure(q): measurement of qubit q

Syntax determines which program expressions can be constructed.

Quantum Type Systems and Static Guarantees

Types can check:

- distinction between classical and quantum data
- ownership and use of qubits
- gate arity and register dimensions
- valid use of measurement
- some forms of illegal duplication
- static resource constraints

Example:

`badCopy(q : Qubit) = (q, q)` $\longrightarrow$ Rejected: the quantum value `q` is used more than once

But, a well-typed program may still implement the wrong algorithm!

A Concrete Example: Bell-State Preparation

Executable representation:

```
qc.h(0)  
qc.cx(0, 1)
```



Abstract circuit:

$\text{Bell} = \text{CNOT} \cdot (\text{H} \otimes \text{I})$



Mathematical semantics:

$\llbracket \text{Bell} \rrbracket |00\rangle = (|00\rangle + |11\rangle) / \sqrt{2}$

Formal verification does not reason only about Python executions.
It requires an abstract representation with a mathematically defined semantics.

Type checking establishes that the program is well formed.
Semantics explains what transformation it performs.

Same Statistics, Different Quantum States

State A

$$|\Phi+\rangle = (|00\rangle + |11\rangle) / \sqrt{2}$$

State B

$$|\Phi-\rangle = (|00\rangle - |11\rangle) / \sqrt{2}$$

Computational-basis measurement

For both states:

$$\text{Pr}[00] = 1/2$$

$$\text{Pr}[11] = 1/2$$

However:

The relative phase is different.

After applying $H \otimes H$:

$$|\Phi+\rangle \rightarrow (|00\rangle + |11\rangle) / \sqrt{2}$$

$$|\Phi-\rangle \rightarrow (|01\rangle + |10\rangle) / \sqrt{2}$$

The same measurement distribution in one basis does not imply the same quantum state.

Interactive Theorem Proving

Proof assistants combine:

- formal mathematical definitions
- human-guided proof development
- automated tactics
- proof checking by a small trusted kernel

What can be certified?

- circuit semantics
- unitarity and circuit identities
- compiler transformations
- correctness of quantum algorithms
- soundness of program logics

Ecosystem	Quantum verification projects
Coq / Rocq	QWIRE, SQIR, VOQC, CoqQ
Isabelle/HOL	Quantum Hoare Logic formalizations
Why3	Qbricks
Lean 4	Emerging circuit and quantum-information libraries

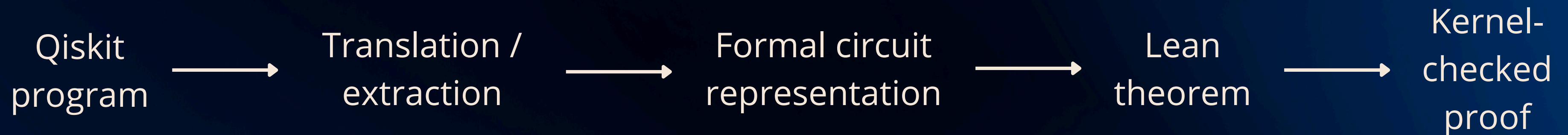
What Is Actually Machine-Checked?

Checked by the proof assistant

- formal definitions are well typed;
- proof terms are checked by the kernel;
- every logical inference follows the formal rules;
- the theorem follows from the stated assumptions.

Still part of the trust boundary

- the formal semantics correctly models the intended gates;
- the Qiskit-to-formal translation is correct;
- external tools return valid results;
- the hardware and noise model match reality.



Verified Compilation

A quantum compiler may perform:

- gate decomposition;
- circuit optimization;
- qubit routing;
- hardware-gate translation;
- removal or reordering of operations.

Semantic preservation
 $\llbracket \text{compile}(C) \rrbracket \equiv \llbracket C \rrbracket$



Circuit Equivalence Checking

Exact equivalence:

$$U_C = U_D$$

Equivalence up to global phase:

$$U_C = e^{i\theta} \cdot U_D$$

Where:

U_C is the transformation implemented by circuit C;

U_D is the transformation implemented by circuit D;

$e^{i\theta}$ is an unobservable global phase.



Quantum Design by Contract

Preconditions

- required number of qubits;
- initial quantum state;
- valid classical parameters;
- ancilla initialized to $|0\rangle$.

Postconditions

- expected final state;
- measurement probabilities;
- minimum success probability;
- ancilla returned to $|0\rangle$;
- required entanglement or separability.

Resource conditions

- oracle calls;
- gate count;
- circuit depth;
- number of ancilla qubits.

Bell-state example

requires: input state = $|00\rangle$

ensures: final state = $(|00\rangle + |11\rangle) / \sqrt{2}$

resources: 1 Hadamard gate
1 CNOT gate

Executable and Machine-Checked Contracts

Executable contracts	Machine-checked contracts
Checked on concrete executions	Proved over a formal semantics
Simulation or hardware runs	Symbolic reasoning
May use sampled measurements	Proof obligations
Useful for debugging	Useful for certification
Limited to tested instances	May cover entire input families

Passed tests $\neq$ universal correctness

Research Gap and Working Hypothesis

Existing approaches

Executable contracts:

- close to practical quantum code;
- useful for simulation and debugging;
- generally limited to selected executions.

Equivalence checkers:

- verify circuit transformations;
- do not normally prove the full algorithmic specification.

Formal verification frameworks:

- provide strong semantic guarantees;
- often use dedicated formal languages;
- may be separated from mainstream development environments.

Research gap

The literature reviewed in this work offers limited end-to-end integration between Qiskit circuit-building programs, executable contracts, circuit equivalence checking, and Lean proof obligations.

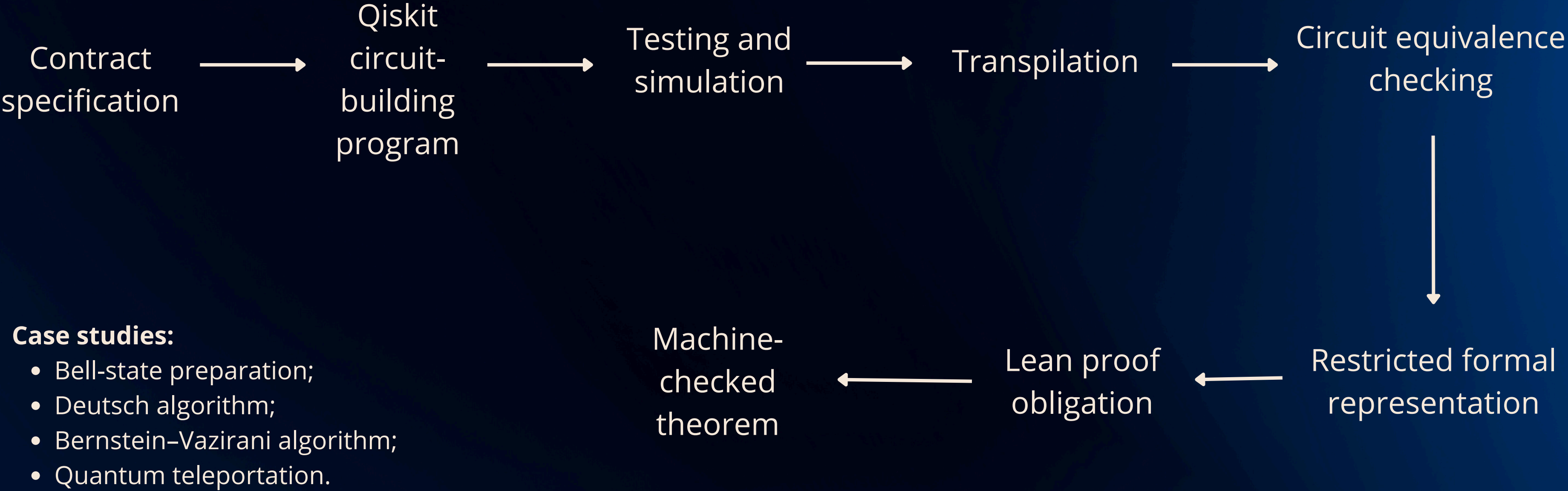
Research question

Can a restricted fragment of Qiskit be connected to a formal semantics and to contracts verified in Lean 4?

Working hypothesis

An end-to-end prototype is feasible for a small gate set, parametric circuit builders, and final measurements.

Exploratory Prototype



Conclusions and Future Direction

Main conclusions

- Quantum software validation is not a single problem.
- Type systems provide static safety, not algorithmic correctness.
- Probabilistic verification provides useful quantitative foundations.
- Quantum semantics must also represent phase, interference and entanglement.
- Testing and simulation provide evidence, not universal proofs.
- Equivalence checking protects circuit transformations.
- Proof assistants can certify semantic properties.
- Contracts may connect executable programs and formal specifications.

Future direction

- Develop a restricted contract language and a Qiskit-to-Lean proof-obligation pipeline for parametric quantum circuits.

References

[1] C. A. R. Hoare, “ An Axiomatic Basis for Computer Programming ” Communications of the ACM, 12(10), 576–580, 1969	[7] M. Ying, “ Floyd–Hoare Logic for Quantum Programs ” ACM Transactions on Programming Languages and Systems, 33(6), Article 19, 2011
[2] M. Kwiatkowska, G. Norman, and D. Parker, “ PRISM 4.0: Verification of Probabilistic Real-Time Systems ” CAV, 2011	[8] J. Paykin, R. Rand, and S. Zdancewic, “ QWIRE: A Core Language for Quantum Circuits ” POPL, 846–858, 2017
[3] C. Hensel, S. Junges, J.-P. Katoen, T. Quatmann, and M. Volk, “ The Probabilistic Model Checker Storm ” International Journal on Software Tools for Technology Transfer, 2022	[9] K. Hietala, R. Rand, S.-H. Hung, X. Wu, and M. Hicks, “ A Verified Optimizer for Quantum Circuits ” Proceedings of the ACM on Programming Languages, 5(POPL), Article 37, 2021
[4] P. Schröder, K. Batz, B. L. Kaminski, J.-P. Katoen, and C. Matheja, “ A Deductive Verification Infrastructure for Probabilistic Programs ” Proceedings of the ACM on Programming Languages - OOPSLA, 2023.	[10] C. Chareton, S. Bardin, F. Bobot, V. Perrelle, and B. Valiron, “ An Automated Deductive Verification Framework for Circuit-Building Quantum Programs ” CAV, 2021
[5] A. Javadi-Abhari et al., “ Quantum Computing with Qiskit ” arXiv:2405.08810, 2024	[11] T. Peham, L. Burgholzer, and R. Wille, “ Equivalence Checking of Quantum Circuits with the ZX-Calculus ” IEEE Journal on Emerging and Selected Topics in Circuits and Systems, 12(3), 662–675, 2022
[6] C. Chareton, S. Bardin, D. Lee, B. Valiron, R. Vilmart, and Z. Xu, “ Formal Methods for Quantum Programs: A Survey ” arXiv:2109.06493, 2021	[12] M. Yamaguchi and N. Yoshioka, “ Design by Contract Framework for Quantum Software ” IEEE/ACM Workshop on Quantum Software Engineering, 24–25, 2023